

TEMA: MODELADO DE ANÁLISIS CON PLANTUML (CU0-0 A CU0-3)

CASO DE ESTUDIO: SISTEMA BOCATTO

1. REQUISITOS FUNCIONALES DE NIVEL 0

Basados en la ERS de Bocatto, se identifican los siguientes requerimientos para el desarrollo del taller:

Código	Requisito Funcional	Actor Principal	Caso de Uso
RF-00	Acceso seguro mediante usuario y clave.	Usuario (General)	CU0-0 Autenticar en el sistema
RF-01	Registro de pedidos directamente en la mesa.	Mesero	CU0-1 Tomar pedido en mesa
RF-02	Toma de pedidos vía telefónica para delivery.	Operador / Recepcionista	CU0-2 Tomar pedido telefónico
RF-03	Autogestión de pedido mediante código QR.	Cliente	CU0-3 Realizar pedido por QR

2. DIAGRAMA DE CASOS DE USO DE ANÁLISIS

Representación del límite del sistema y los actores involucrados en los procesos de pedido.

```
@startuml
```

```
left to right direction
```

```
skinparam packageStyle rectangle
```

```
actor "Usuario" as U
```

```
actor "Mesero" as M
```

```
actor "Operador" as O
```

```
actor "Cliente" as C
```

```
rectangle "Sistema Bocatto - Gestión de Pedidos" {
```

```
U -- (CU0-0: Autenticar en el sistema)
```

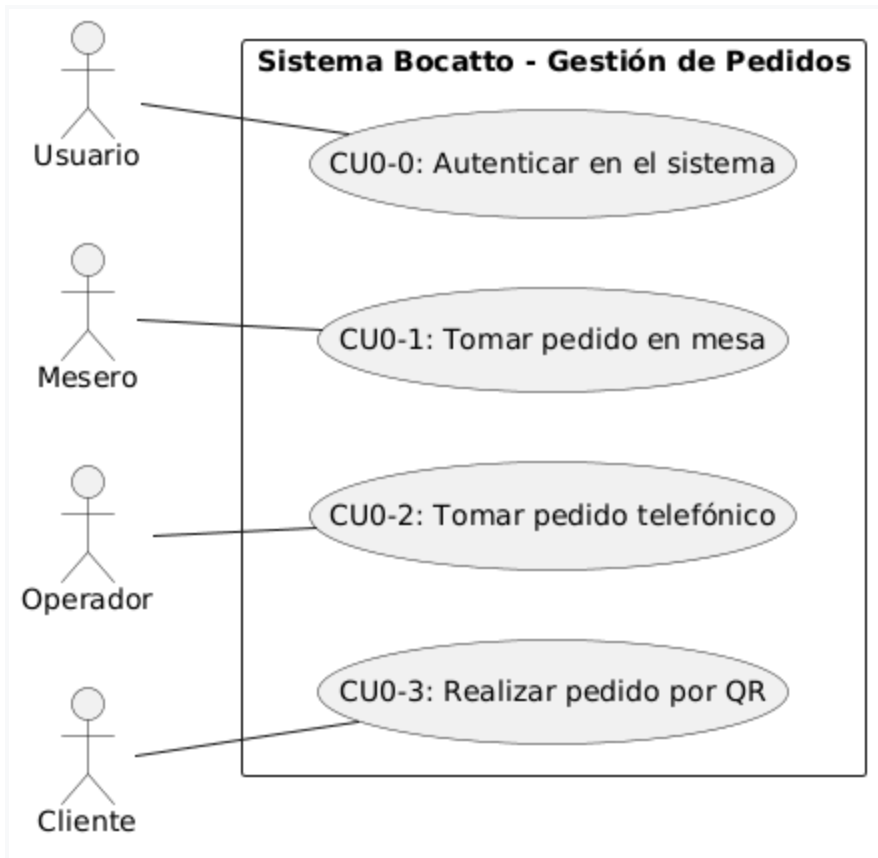
```
M -- (CU0-1: Tomar pedido en mesa)
```

```
O -- (CU0-2: Tomar pedido telefónico)
```

```
C -- (CU0-3: Realizar pedido por QR)
```

```
}
```

```
@enduml
```



3. ESPECIFICACIÓN RESUMIDA DE CASOS DE USO

Tipo de clase	Responsabilidad en el análisis	Clases propuesta	Relación con los casos de uso
Boundary / Frontera	Recibe datos del actor y muestra resultados.	FormularioLogin	Interfaz para el usuario para acceder al sistema mediante usuario y clave.
		InterfazPedidoMesa	Interfaz para el mesero para registrar un pedido para una mesa.
		InterfazPedidoTel	Interfaz para el cliente para realizar un pedido por delivery con un número telefónico.
		AppMovilQR	Interfaz para el cliente para realizar un pedido usando un código QR

			localizado en la mesa.
Control	Coordina la lógica de cada caso de uso.	GestorAutenticacion	Valida que las credenciales ingresadas sean válidas.
		ControlRegistroPedidos	Administra los pedidos registrados por los usuarios.
Entity / Entidad	Representa la información del dominio.	Usuario	Superentidad que incluye a los clientes, meseros y operadores.
		Pedido	Conjunto de platos solicitados por los clientes.
		Plato	Entidad que representa uno de los diferentes platos disponibles.
		Cliente	Entidad que realiza pedidos.
Repositorio / Colección	Conserva y recupera registros.	RepositorioUsuarios	Permite validar las credenciales de los usuarios para brindar acceso a cada uno a su respectiva interfaz.

4. DIAGRAMA DE CLASES DE ANÁLISIS (BCE)

Identificación de objetos de Frontera, Control y Entidad para asegurar la trazabilidad estructural.

@startuml

' --- FRONTERAS (Boundary) ---

class FormularioLogin <<boundary>>

class InterfazPedidoMesa <<boundary>>

class InterfazPedidoTel <<boundary>>

class AppMovilQR <<boundary>>

' --- CONTROLES (Control) ---

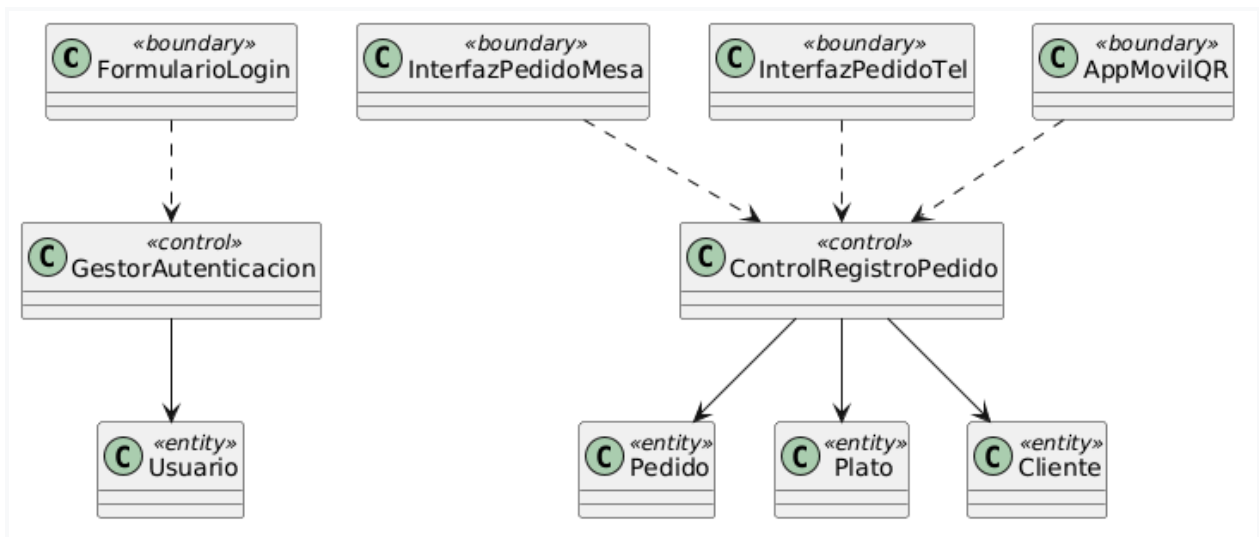
```
class GestorAutenticacion <<control>>
class ControlRegistroPedido <<control>>
```

' --- ENTIDADES (Entity) ---

```
class Usuario <<entity>>
class Pedido <<entity>>
class Plato <<entity>>
class Cliente <<entity>>
```

' Relaciones de dependencia y asociación

```
FormularioLogin ..> GestorAutenticacion
InterfazPedidoMesa ..> ControlRegistroPedido
InterfazPedidoTel ..> ControlRegistroPedido
AppMovilQR ..> ControlRegistroPedido
ControlRegistroPedido --> Pedido
ControlRegistroPedido --> Plato
GestorAutenticacion --> Usuario
ControlRegistroPedido --> Cliente
@enduml
```



5. MATRIZ DE RELACIÓN: CASOS DE USO, CLASES Y

SECUENCIAS

Caso de uso	Boundary	Control	Entity / Repositorio	Diagrama de secuencia esperado
Autenticar en el Sistema	FormularioLogin Captura usuario y contraseña	GestorAutenticacion Valida que las credenciales ingresadas sean válidas	Usuario Usuario ingresa sus credenciales al sistema	Secuencia: ingresar datos → validar → identificar rol → habilitar acceso a las funcionalidades permitidas
Registro de pedidos directamente en la mesa.	InterfazPedidoMesa Captura número de mesa, productos seleccionados, cantidad por producto y observaciones del pedido (opcional).	ControlRegistroPedidos Administra los pedidos registrados por los usuarios	Usuario, Cliente, Pedido, Plato Usuario registra un Pedido con Platos para Cliente	Ingresar Mesa → mostrar menú → seleccionar productos → confirmar → registrar pedido → generar ID → confirmar
Registrar Pedido Telefónico	InterfazPedidoTel Captura productos seleccionados, cantidad por producto, observaciones del pedido (opcional), datos del cliente para contacto, dirección de entrega.	ControlRegistroPedidos Administra los pedidos registrados por los usuarios	Usuario, Cliente, Pedido, Plato Usuario registra un Pedido con Platos para Cliente	Realizar llamada → mostrar menú → seleccionar productos → confirmar → registrar pedido → generar ID → confirmar
Autoregistro de pedidos por QR	AppMovilQR Captura QR de mesa (identificador de mesa), productos seleccionados, cantidad por producto, observaciones del pedido (opcional).	ControlRegistroPedidos Administra los pedidos registrados por los usuarios	Pedido, Plato, Cliente Cliente registra un Pedido con Platos para sí mismo	Escanear QR → mostrar menú → seleccionar productos → confirmar → registrar pedido → generar ID → confirmar

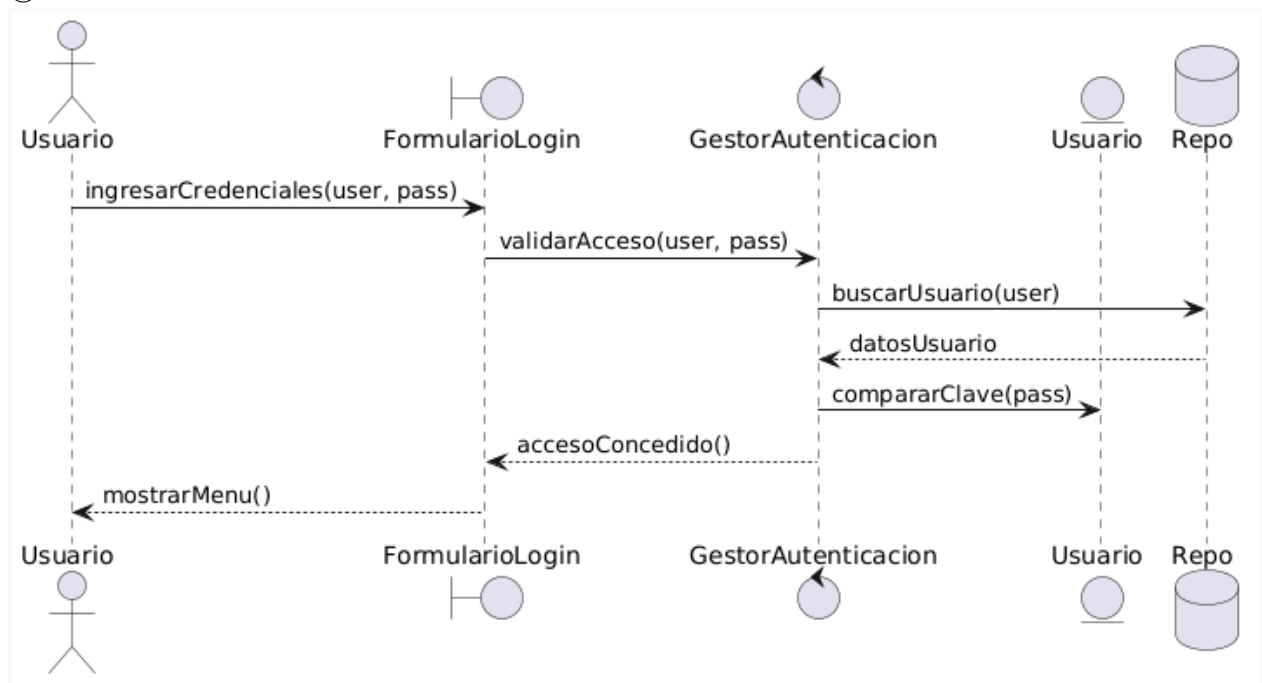
6. DIAGRAMAS DE SECUENCIA DE ANÁLISIS

6.1 CU0-0: Autenticar en el sistema

@startuml
actor Usuario
boundary FormularioLogin
control GestorAutenticacion
entity UsuarioE as "Usuario"
database RepositorioUsuarios as "Repo"

Usuario -> FormularioLogin : ingresarCredenciales(user, pass)
FormularioLogin -> GestorAutenticacion : validarAcceso(user, pass)
GestorAutenticacion -> RepositorioUsuarios : buscarUsuario(user)
RepositorioUsuarios --> GestorAutenticacion : datosUsuario
GestorAutenticacion -> UsuarioE : compararClave(pass)
GestorAutenticacion --> FormularioLogin : accesoConcedido()
FormularioLogin --> Usuario : mostrarMenu()

@enduml

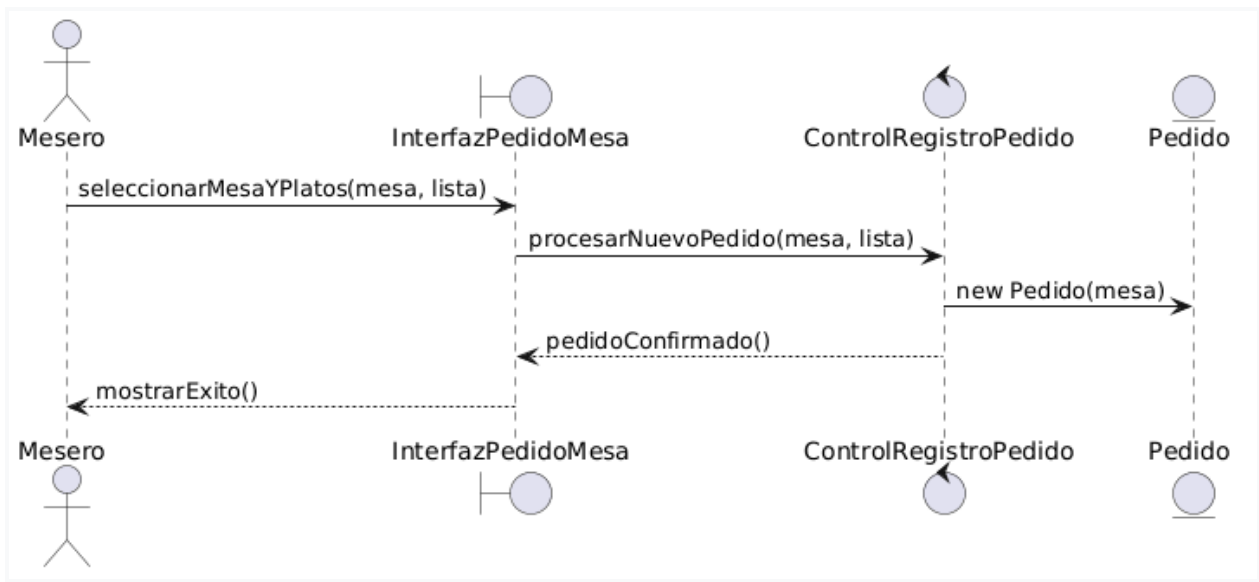


6.2 CU0-1: Tomar pedido en mesa

@startuml
actor Mesero
boundary InterfazPedidoMesa

control ControlRegistroPedido
entity Pedido as "Pedido"

Mesero -> InterfazPedidoMesa : seleccionarMesaYPlatos(mesa, lista)
InterfazPedidoMesa -> ControlRegistroPedido : procesarNuevoPedido(mesa, lista)
ControlRegistroPedido -> Pedido : new Pedido(mesa)
ControlRegistroPedido --> InterfazPedidoMesa : pedidoConfirmado()
InterfazPedidoMesa --> Mesero : mostrarExito()
@enduml

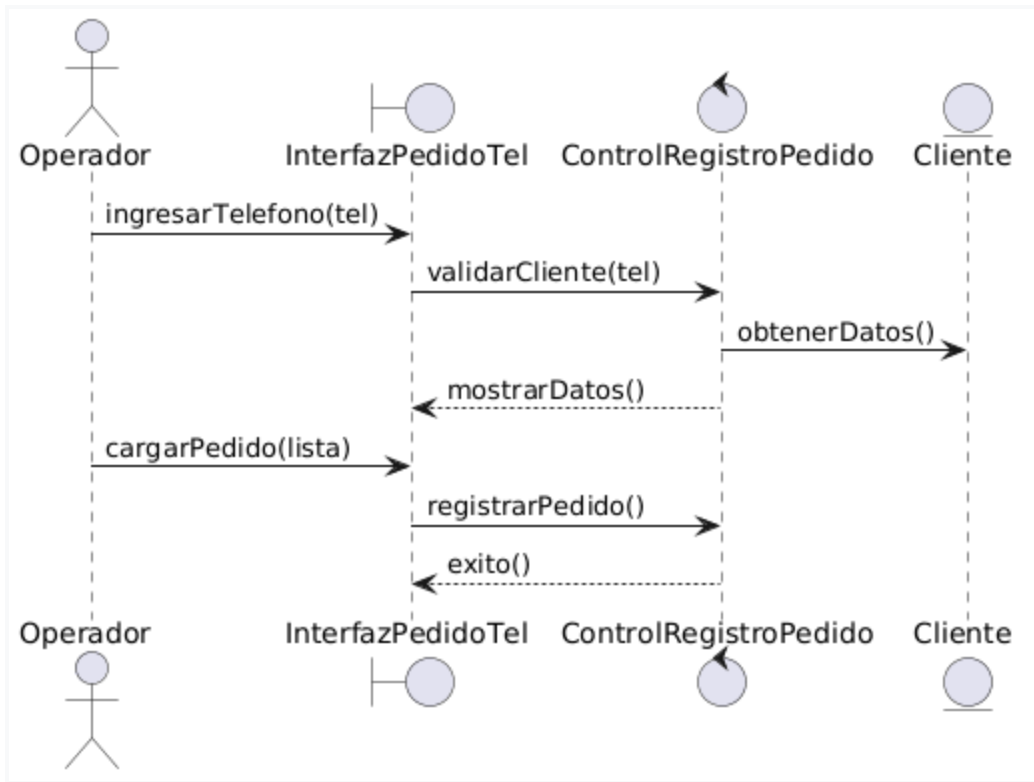


6.3 CU0-2: Tomar pedido telefónico

@startuml
actor Operador
boundary InterfazPedidoTel
control ControlRegistroPedido
entity Cliente as "Cliente"

Operador -> InterfazPedidoTel : ingresarTelefono(tel)
InterfazPedidoTel -> ControlRegistroPedido : validarCliente(tel)
ControlRegistroPedido -> Cliente : obtenerDatos()
ControlRegistroPedido --> InterfazPedidoTel : mostrarDatos()
Operador -> InterfazPedidoTel : cargarPedido(lista)

InterfazPedidoTel -> ControlRegistroPedido : registrarPedido()
ControlRegistroPedido --> InterfazPedidoTel : exito()
@enduml

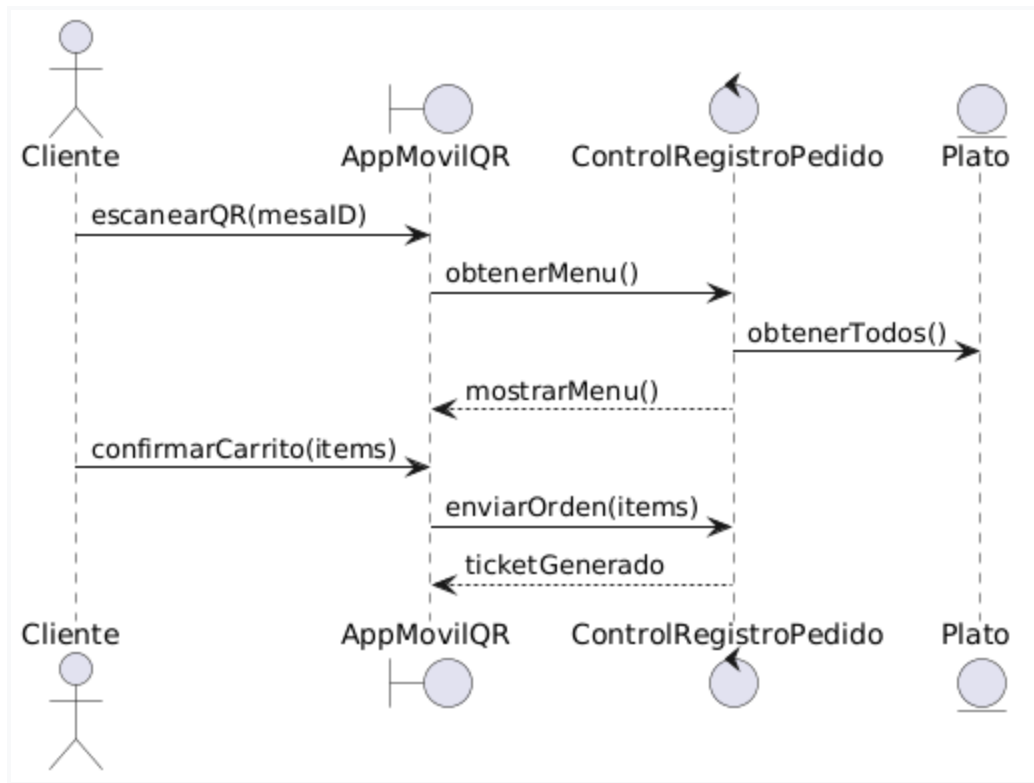


6.4 CU0-3: Realizar pedido por QR

@startuml
actor Cliente
boundary AppMovilQR
control ControlRegistroPedido
entity Plato as "Plato"

Cliente -> AppMovilQR : escanearQR(mesaID)
AppMovilQR -> ControlRegistroPedido : obtenerMenu()
ControlRegistroPedido -> Plato : fetchAll()
ControlRegistroPedido --> AppMovilQR : mostrarMenu()
Cliente -> AppMovilQR : confirmarCarrito(items)
AppMovilQR -> ControlRegistroPedido : enviarOrden(items)
ControlRegistroPedido --> AppMovilQR : ticketGenerado

@enduml



7. MATRIZ DE TRAZABILIDAD (RF - CU - CLASES)

RF	Caso de Uso	Objetos (BCE)	Prioridad
RF-00	CU0-0	FormularioLogin, GestorAutenticacion, Usuario	Alta
RF-01	CU0-1	InterfazPedidoMesa, ControlRegistroPedido, Pedido, Plato	Alta
RF-02	CU0-2	InterfazPedidoTel, ControlRegistroPedido, Cliente, Pedido	Media
RF-03	CU0-3	AppMovilQR, ControlRegistroPedido, Pedido, Plato	Alta

8. CONCLUSIÓN TÉCNICA

La implementación del modelo de análisis para el Sistema Bocatto demuestra una arquitectura robusta y escalable mediante el uso de las siguientes estrategias técnicas:

- **Aplicación del Patrón BCE (Boundary-Control-Entity):** El uso de este patrón garantiza una clara separación de responsabilidades. Las interfaces (Boundary) como **InterfazPedidoMesa** o **AppMovilQR** se encargan exclusivamente de la interacción con el actor, mientras que la lógica de negocio se centraliza en objetos de control como **ControlRegistroPedido**, lo que facilita el mantenimiento futuro del código.
- **Trazabilidad Integral de Requisitos:** Existe una alineación directa desde los requisitos funcionales (RF-00 a RF-03) hasta los diagramas de secuencia y clases. Esta matriz de trazabilidad asegura que cada funcionalidad solicitada por el negocio tenga un respaldo estructural y dinámico en el diseño del sistema.
- **Versatilidad en la Captura de Pedidos:** El sistema está diseñado para gestionar el núcleo del negocio (pedidos) desde múltiples frentes: presencial (mesero), telefónico (operador) y digital (cliente vía QR). Técnicamente, esto se resuelve mediante el polimorfismo de fronteras que convergen en un controlador común, optimizando la reutilización de la lógica de procesamiento de órdenes.
- **Seguridad y Gestión de Datos:** La inclusión del caso de uso CU0-0: Autenticar en el sistema y el uso de un RepositorioUsuarios garantiza que el acceso a las funciones críticas esté protegido mediante un flujo de validación definido en los diagramas de secuencia.
- **Eficiencia en el Modelado con PlantUML:** El uso de herramientas de "Diagram as Code" (PlantUML) permite una documentación técnica ágil, versionable y estandarizada, lo que reduce la ambigüedad entre el análisis y la posterior fase de implementación.

En conclusión, el modelo presentado no solo cubre las necesidades operativas de Bocatto, sino que establece una base técnica sólida, modular y fácil de auditar para el equipo de desarrollo.